Determining Metaheuristic Similarity Using Behavioral Analysis

Lauren Hayward and Andries Engelbrecht, *Senior Member, IEEE*

***Abstract*—Many nature-inspired metaheuristics have been published, with claims of originality based on the metaphor that inspired the algorithm. Rarely is empirical evidence given to show algorithmic originality. In order to provide an easy and computationally cheap approach to characterize algorithm search behavior, a suite of 20 behavioral characteristics is proposed. This behavioral characteristic suite allows for the search behavior of an algorithm to be quantified without manual inspection. By doing so, behavioral novelty of any given algorithm may be determined by comparing the behavioral characteristics to those of well-known metaheuristics. To illustrate this use, and to evaluate whether metaheuristics are behaviorally distinct, a host of metaheuristics is run on various benchmark functions. To evaluate behavioral similarity across all problems, while acknowledging behavior to be problem dependant, a novel method is proposed. In addition to this method, new behavioral characteristics are also proposed. The behavioral vectors generated for each benchmark function are clustered. The relationships and trends present in the different clusters are summarized by creating a pair-wise matrix for every metaheuristic pair, which tallies the number of times that the pair are found within the same cluster. The tallies are then analyzed in order to make inference regarding the distinctness of any metaheuristic's behaviors, across many different benchmark functions. The analysis finds that the range of unique search behaviors is small and that most metaheuristics share their behaviors with most other metaheuristics. The analysis also identifies both unique algorithms, as well as algorithms which have no unique behaviors.**

***Index Terms*—Data analysis, data engineering, evolutionary computation, genetic algorithms, particle swarm optimization, pattern analysis.**

I. Introduction

MANY algorithms have been published within the nature-inspired optimisation field, each drawing inspiration from some natural phenomenon. A total of 1222 unique publications were found within the field of metaheuristics when surveyed by Hussain et al. [18], and there also exists a community of researchers maintaining a "bestiary" list of nature-inspired algorithms [4]. Rarely are concrete explanations given as to how and why these algorithms are new, other than the natural analogy upon which they are based [48]. So-called improvements to existing algorithms are proposed, supported only by benchmark results [10]. Such improvements are rarely accompanied by empirical analysis or analogy-agnostic explanation [17], prompting various authors to speak out against the poor standard of nature-inspired metaheuristics [2]. It is clear that there lacks an accessible method through which to classify algorithm behavior.

It is known that the behavior of an optimisation algorithm is fitness landscape dependant [11]. Additionally, the control parameters used in an instance of an algorithm have been shown to vastly influence the behavior of the algorithm [6]. These factors make wide scale comparison between algorithms impractical. In order to address the inaccessibility of wide scale algorithm behavioral characterization, a benchmarking suite of behavioral characteristics is proposed. The suite characterizes algorithm search behavior in terms of how the algorithm explores and exploits, the localities in the search space that are explored, the manner in which the algorithm communicates, and the evaluation effort that is expended. The intention in doing so is to provide an accessible and computationally cheap method by which to quantify the search behavior of an algorithm instance, where an algorithm instance is the unique combination of metaheuristic, control parameters, and benchmark function. This can be used to factorize groups of algorithm instances, which allows for conclusions to be made regarding the uniqueness of an algorithm's search behaviors.

A wide range of nature-inspired metaheuristics are classified using the behavioral characteristic benchmarking suite, across a range of problems and control parameter sets. The results generated for each benchmark function are clustered using density-based spatial clustering of applications with noise (DBSCAN) [12]. The similarity between the behavior of metaheuristics is determined by tallying the number of times that each possible pair of metaheuristics is found in the same cluster. These tallies are used to create a pair-wise matrix of behavior similarity, named a frequency map. The results in the form of the frequency map are explored and notable similarities and dissimilarities between algorithms are discussed.

This article makes the following significant contributions.

1) Eight new behavioral characteristics, which characterize the search behavior of a metaheuristic in terms of how it explores and how it exploits. These characteristics are

Manuscript received 27 February 2023; revised 17 August 2023; accepted 16 December 2023. Date of publication 25 December 2023; date of current version 31 January 2025. This article was approved by Associate Editor T. Bäck. *(Corresponding author: Andries Engelbrecht.)*

Lauren Hayward is with the Computer Science Division, Stellenbosch University, Stellenbosch 7602, South Africa (e-mail: haywardlau@gmail.com).

Andries Engelbrecht is with the Department of Industrial Engineering and Computer Science Division, Stellenbosch University, Stellenbosch 7602, South Africa, and also with the Center for Applied Mathematics and Bioinformatics, Gulf University of Science and Technology, Hawally 32093, Kuwait (e-mail: engel@sun.ac.za).

This article has supplementary downloadable material available at https://doi.org/10.1109/TEVC.2023.3346672, provided by the authors.

Digital Object Identifier 10.1109/TEVC.2023.3346672

named the prime population spread (PPS), the accuracy rate of change (types A and B), the prime fitness value (PFV), the convergence rate of change (CRoC) (types A and B), and the locality rate of change (types A and B).
2) A behavioral benchmarking suite consisting of 20 behavioral characteristics.
3) A novel method of analyzing and visualizing clustering results, named a frequency map.
4) Conclusions regarding the lack of diversity in search behaviors exhibited by metaheuristics.

The remainder of this article is structured as follows: first, existing methods of characterizing algorithm search behavior are detailed in Section II. Following on this, the behavioral characteristic benchmarking suite is introduced in Section III. This is followed by an explanation of the empirical procedure in Section IV. Results are discussed in Section V and finally Section VI concludes.

II. Background

There are different categories of algorithmic equivalence: 1) structural equivalence, such as genetic algorithms, that share recombination and mutation operators; 2) performance equivalence, such as algorithms, exhibiting the same efficiency, where efficiency refers to all factors of performance; or 3) behavioral equivalence, such as algorithms, exhibiting the same search patterns. In order to quantify behavioral similarity between algorithms, a suite of characteristics is proposed, through which the search-behavior of an algorithm may be footprinted. Much like the existing work on footprinting problem space, i.e., *exploratory landscape analysis* [33], an array of indicators covering various characteristics of an algorithm's search behavior is used to factorize algorithm instances into groups. The search behavior of an algorithm is the manner and methods through which the algorithm searches the fitness landscape. An instance of an algorithm is the unique combination of metaheuristic, control parameter set, and fitness landscape upon which the algorithm is applied.

This section provides an explanation of the metaheuristic model used, as well as a review of existing metaheuristic behavioral analysis, as background on the measures defined in the behavior characterization benchmarking suite to follow in Section III.

A. Metaheuristic Model

The metaheuristics in this article fall under the swarm intelligence paradigm, such as particle swarm optimisation [27], and evolutionary computation, such as genetic algorithms [16], genetic programming [29], and evolutionary programming [14]. In order to compare metaheuristics using different terminology, generalized terms are preferred over specific terms. Candidate solutions are considered to be "individuals" and the collection of candidate solutions is considered to be the "population." All optimisation in this article is assumed to be a minimizing strategy, and for the purposes of investigation all problem spaces are well defined with known boundaries and optima, which do note change over time.

B. Existing Behavioral Analysis

Many authors have introduced methods with which to characterize specific algorithmic behaviors. Such behaviors include performance [34], [47], [55], diversity [3], [7], exploration and exploitation [55], the locality wherein the search takes place [7], [43], [55], and the interaction among individuals within the algorithm [44]. The following existing methods of characterization are included in the behavioral characteristics to follow, or are the root inspiration for a new method of behavioral characterization proposed in this article.

For the characterization of problem space, an estimated running time (ERT)-based ranking system is created by Mersmann et al. [34], which allows for the grouping of algorithms according to their performance on problems of varying complexity.

Three methods with which to describe the search behavior of ant colony optimisation algorithms are presented by Zecchin et al. [55]. The methods characterize the efficacy of the search, the extent to which the algorithm explores, and the search behavior with regard to feasible and infeasible space. To characterize the efficacy of the algorithm, where being effective means finding optimal or near optimal solutions, the authors propose tracking the change in the best-objective function value, and tracking the change in the distance from the global optima. For the distance calculation, various methods are suggested, such as Euclidian distance for continuous space and Hamming distance for discrete space. To characterize exploration, tracking the change in the mean distance between all individuals in the search space is suggested. The characterization of the search behavior with regard to feasible and infeasible space is done by tracking the percentage of the search spent in feasible space.

For the characterization of the performance of local search, Schuurmans and Southey defined three characteristics [47]: 1) "Depth" as the number of unsatisfied clauses, which indicates how deep within the search space the local search remains; 2) "Mobility" referring to how rapidly the local search moves through the search space, which is measured by calculating the Hamming distance between candidate solutions; and 3) "Coverage" to measure how systematically search space is searched by the local search algorithm. This is calculated by measuring how rapidly the largest gap between two explored points in the search space in terms of Hamming distance is reduced.

Bosman and Engelbrecht introduced a measure of diversity, i.e., "diversity rate of change (DRoC)," to characterize and compare algorithms in terms of the rate at which they converge [3]. The characteristic is calculated by measuring the average distance of each individual from the center of mass of the population every iteration. By taking a two-piecewise linear approximation of the function obtained from this temporal data, the measure of diversity is defined to be the first slope of the linear approximation.

Ochoa et al. [43] created the search trajectory network (STN) with the goal of being able to characterise the search trajectory that individuals undertake during optimisation. The STN uses a graph model to reduce a search landscape into

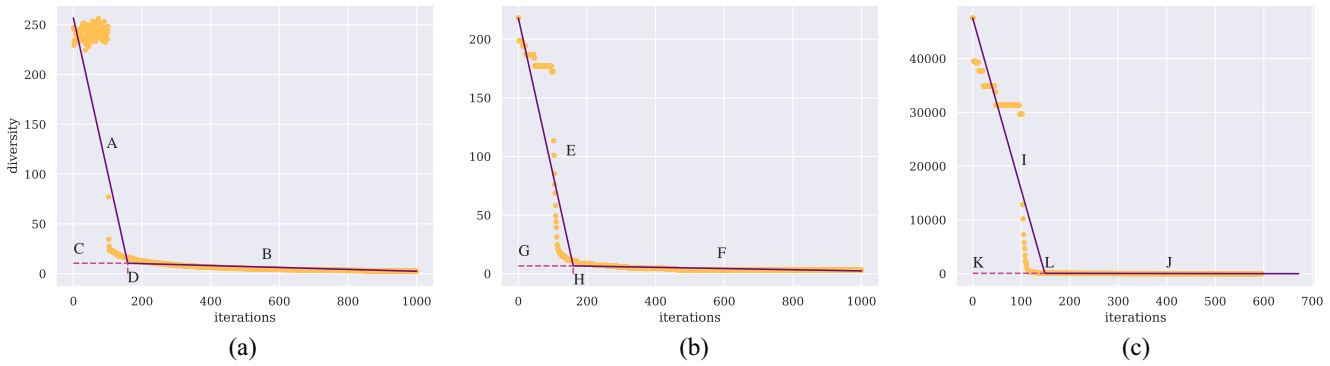

Fig. 1. Interaction between the data, two-piecewise linear approximation, and the knee-point from changes in diversity, Euclidean distance, and objective function value. (a) Construction of characteristics from changes in diversity. (b) Construction of characteristics from changes in the distance from the global minima. (c) Construction of characteristics from changes in the fitness difference from the global minima.

"locations" - of size relative to the total landscape size - which are subsets of the greater landscape. Each location has a predetermined objective function value, and any individual is in one and only one location at any given time. The model then maps search trajectories through real search space to a sequence of locations, which are more likely to be shared with other individuals than when comparing unique, continuous, d-dimensional points in space. This results in a directed graph that represents individuals' journeys through d-dimensional search space, which is otherwise too complex to visualize.

Interaction networks (INs) were created by Oliveira et al. [44] to illustrate and characterise the way in which individuals communicate. Social interaction is paramount to the workings of many metaheuristics and all systems which contain parts that can interact, can be modeled as a social network. INs model the social connectivity between individuals, and can be expanded to most optimisation heuristics by defining what "communication" means per algorithm.

III. Characteristic Benchmarking Suite

This section introduces the characteristic benchmarking suite, which is comprised of both old and new methods of characterizing algorithm search behavior. Many of the methods in previous works characterizing algorithmic behavior are defined per iteration rather than a single value characterizing the overall behavior. To address this, the measures used in the benchmarking suite are defined for the whole duration of an algorithm's search.

The benchmarking suite is divided into subgroups: "Exploration," "Exploitation," "Locality in the Search Space," "Communication," and "Evaluation Effort." The rest of this section defines the indicators of each group, as well as what data is collected and the process of calculating the indicators. Unless explicitly stated, an indicator is newly developed for this analysis.

A. Exploration and Exploitation

Considering static optimisation problems, there are two phases in the optimisation process: 1) the first is exploration, which is the phase in which the individuals explore large parts of the search space in order to find a promising locality. Once the population starts to converge heavily, this phase is over and 2) the second phase in the optimisation process is exploitation, in which the individuals explore a small promising locality until the search process is ended. As exploration and exploitation are closely linked, so are the indicators under "Exploitation" and "Exploitation." In order to calculate the indicators relating to exploration and exploitation, the following data is collected.

Every iteration, the state of the search process is reflected by measuring the diversity of the population, and the level of accuracy of the population. Here, "diversity" is the difference with regards to solution vectors in a population's individuals, which reflects the diversity of potential solutions. Bosman and Engelbrecht's method of calculating diversity is used, where the average distance from the population centroid is measured [3]. "Accuracy" refers to how near the algorithm is to the global optima. This is measured using methods created by Zecchin et al. [55], where the difference between the best-current solution and the global optima is measured, both in terms of Euclidean distance and difference in objective function value.

The three types of data points collected are processed in the following way. All three are graphed as temporal data, where the unit of time is iterations. A two-piecewise linear approximation is applied to the three graphs. On each graph, this creates four points of interest: 1) the slope of the first line; 2) the slope of the second line; as well as 3) the x- and y-coordinates of the knee-point. These points of interest are labeled as "A," "B," "C," and "D" on Fig. 1(a), which graphs the diversity temporal data. Similar graphs for accuracy can be seen labeled "E," "F," "G," and "H" in Fig. 1(b), for the Euclidean distance measurement of accuracy, and also labels "I," "J," "K," and "L" in Fig. 1(c), which is the objective function value measurement of accuracy. The points of interest identified in these graphs are assigned to five indicators of exploration and four indicators of exploitation.

The first indicator under "Exploration" is Bosman and Engelbrecht's DRoC [3], which is the slope "A" in Fig. 1(a). DRoC characterizes the rate at which the search transitions from exploration to exploitation. The second indicator is the diversity at the end of the exploration phase, which is

labeled "C" in Fig. 1(a). The value of diversity at the end of exploration represents the spread of the population when a promising locality is identified. This indicator is referred to as the PPS. The third indicator is the slope at label "E" in Fig. 1(b), and is referred to as accuracy rate of change type A (ARoC-A). ARoC-A characterizes the rate at which the algorithm finds a promising locality. The fourth indicator is the slope at label "I" in Fig. 1(c), and is referred to as accuracy rate of change type B (ARoC-B). ARoC-B characterizes the rate of improvement during exploration. The final indicator under "Exploration" is the difference in objective function value from the global minimum at the end of the exploration phase, which is labeled "K" in Fig. 1(c). The difference between the current best-objective function value and the global minimum as indicated by the point "K" represents the fitness strength required to change from exploration to exploitation. This is referred to as the PFV.

The first indicator under "Exploitation" is the CRoC, and indicates the rate at which the individuals converge on a single point. CRoC is the slope at label "B" in Fig. 1(a). The second indicator is the locality rate of change type A (LRoC-A), and indicates the speed at which the algorithm is able to find the best value in a small area. LRoC-A is the slope at label "J" in Fig. 1(c). The last indicator under "Exploitation" is the distance from the global minimum at the start of exploitation, and is the value at label "G" in Fig. 1(b). The distance from the global minimum at point "C" represents the radius of the promising locality to be exploited. This is referred to as the exploitation radius (ER).

B. Locality in the Search Space

In order to characterise the areas of the search space investigated by an optimisation algorithm, without requiring detailed knowledge of the search space, the characteristics chosen reflect trends rather than focusing on the objective space of known benchmark problems. To do so, a STN [43] is created, mapping the search trajectories of individuals during the search process.

The three characteristics chosen to reflect the localities visited in the search space are metrics provided by the STN. The first of the three characteristics is called the "nshared" value, which denotes how many of the nodes in the graph are shared by individuals. The magnitude of nshared indicates whether there were locations in the search space that were very attractive to a wide array of individuals. The second characteristic is called the "ntotal" value, which indicates how widely the algorithm explored the search space. The last characteristic is called the "nend" value. The magnitude of nend indicates how frequently individuals end up in suboptimal space. A series of locations is classified as suboptimal when an individual moves along the series of locations, only to later backtrack along the same series of locations.

C. Communication

In order to characterise communication, defined as interaction between individuals, a single IN [44] is created that spans all interactions across the entire search process. An interaction is defined as knowledge transfer between individuals. The weights of the edges in the IN, which equals the number of times two nodes have interacted, are normalized to total 1, so that results are comparable between algorithms and benchmark functions. The IN is then used to infer individual characteristics.

The first characteristic is the sum of the normalized edge-weights of the final solution node. The size of the sum reflects how widely the final best-individual influences the population, and the strength of the influence applied. This is referred to as best strength of influence (BSoI).

The second characteristic is generated by graphing the total disjoint subgraphs of the network. The total disjoint subgraphs increase when edges are removed. To observe this as a change, edges are removed in ten equal steps from least weighted edge to most weighted edge. Any edge with a weight lower than the weight being considered is removed. This change in total number of disjoint subgraphs is graphed, and a linear approximation is taken. The slope of the linear approximation of the graph characterizes how widely the communication within the population was distributed. This is referred to as the information rate of flow (IRoF).

Two more characteristics to reflect the communication amongst individuals are taken from the metrics provided by the STN created in Section III-B. The first is called the "best-strength" value, and indicates how attractive and reachable the global minimum point is. The magnitude of best-strength indicates how many individuals receive information regarding the global best position. The second characteristic is called the "nbest" value, which tallies the total times the best location is visited, and indicates the effectivity of communication amongst the individuals.

D. Evaluation Effort

The final group of characteristics aims to measure the effort the algorithm puts into solving the problem at hand.

The first characteristic chosen to do this, is the total number of objective function evaluations made during the search process. The number of objective function evaluations reflects the computational effort used by the algorithm, as well as how the algorithm functions.

The next three characteristics measure the ERT according to different factors. The characteristics are taken using the values at labels "D," "H," and "L" in Fig. 1(a)–(c), respectively. These values are referred to as *ERT-Diversity*, *ERT-Accuracy-A*, and *ERT-Accuracy-B*.

The final measure taken is the percentage of time spent in infeasible space, referred to as *%INFEASIBLE*. Again, time is measured as iterations, where a single individual being in infeasible space for an iteration is counted as $(1/n)$ of a single time unit, where n is the size of the population. Time spent in infeasible space indicates wasted search effort. The complement is the percentage of time spent in feasible space.

E. Conclusion

In conclusion, 20 characteristics by which to footprint the search behavior of an optimisation algorithm are identified.

The behavioral characteristics can be split into five groups representing different features of an algorithm. A summary of the characteristics is provided in the supplementary material.

IV. Empirical Procedure

This section details the benchmark functions the metaheuristics are tested upon. Thereafter follows the metaheuristics included in this study, and the final subsection details the procedure of the experiments.

A. Benchmark Functions

It is known that metaheuristic behavior is problem dependant [11]. To this end, it is important to have an array of benchmark functions to allow for the algorithms to exhibit different behaviors [11]. Due to the design decisions made when constructing the characteristic benchmarking suite in Section III, the functions that are chosen must be strictly static. The characteristics assume that a single cycle of exploration and exploitation is followed, which a dynamic environment would not adhere to.

Additionally, the benchmark functions must be chosen for their ability to elicit distinct behaviors from different metaheuristics. This implies that the functions should be dissimilar from one another. For this reason the benchmark functions have been chosen according to Lang and Engelbrecht's guidance [31], which suggests benchmark functions based on covering the widest array of landscape characteristics possible. However, it has been necessary to keep the problems relatively trivial to solve, to reduce the time spent executing experiments. For this reason the entire benchmarking suite proposed in [31] has not been used, and some alternative functions have been chosen.

The benchmark functions used are provided in detail in the supplementary material. All benchmark functions are used with a dimension of 20.

B. Metaheuristics

The metaheuristics implemented were chosen with the following process. First, a list was made of the top cited metaheuristics papers, totaling 127 papers. From this list, priority was given to papers from authors with more than one paper on the list. This was done based on the assumption that there is likely to be behavioral overlap from authors with more than one metaheuristic. Beyond those with priority, the rest of the metaheuristics were selected in a random order. This reduces selection bias. Finally, a number of "baseline" heuristics were implemented. These are to represent the standard types of behaviors found, and include variants of swarm intelligence and evolutionary algorithms.

Table I lists all the baseline metaheuristics implemented. Each of these represents many subversions of itself, such as genetic algorithms with different genetic operators. Under evolutionary programming and genetic algorithms, there are versions which use constant parameters and others which use adaptive parameters. The tags for these are "C-EP", "C-GA", "A-EP", and "A-GA". Under particle swarm optimisation, there are two subtypes. These are the bare bones particle swarm optimiser (B-PSO) [26], and the modified bare bones particle swarm optimiser (MB-PSO) [26].

TABLE I
Baseline Metaheuristics

Title	Authors	Tag	Cite
Evolutionary Programming	Fogel	EP	[14]
Evolutionary Strategy	Rechenberg	ES	[46]
Differential Evolution	Storn, Price	DE	[49]
Genetic Algorithm	Fraser	GA	[16]
Particle Swarm Optimisation	Kennedy, Eberhart	PSO	[27]
Simulated Annealing	Kirkpatrick et al.	SA	[28]

TABLE II
Individual Metaheuristics Implemented

Algorithm	First Author	Tag	Cite
Antlion Optimiser	S. Mirjalili	AO	[35]
Artificial Bee Colony	D. Karaboga	ABC	[19]
Atom Search Optimisation	W. Zhao	ASO	[56]
Bat Algorithm	X-S. Yang	BA	[51]
Cat Swarm Algorithm	S-C. Chu	CSA	[5]
Central Force Optimisation	R. A. Formato	CFO	[15]
Charged System Search	A. Kaveh	CSS	[24]
Chicken Swarm Optimisation	X. Meng	CSO	[32]
Colliding Bodies Optimisation	A. Kaveh	CBO	[23]
Cuckoo Search via Levy Flights	X-S. Yang	CSLF	[54]
Dragonfly Algorithm	S. Mirjalili	DA	[37]
Firefly Algorithm	X-S. Yang	FA	[52]
Flower Pollination Algorithm	X-S. Yang	FPA	[53]
Glowworm Optimisation	K. Krishnanand	GO	[30]
Grey Wolf Optimiser	S. Mirjalili	GWO	[41]
Manta Ray Foraging Optimisation	W. Zhao	MRFO	[57]
Marriage in Honey Bees	H. A. Abbass	MHB	[1]
Moth Flame Optimisation	S. Mirjalili	MFO	[36]
Multi-verse Optimiser	S. Mirjalili	MVO	[40]
Salp Swarm Algorithm	S. Mirjalili	SSA	[39]
Seagull Optimisation Algorithm	G. Dhiman	SOA	[9]
Shuffled Frog-leaping algorithm	M. Eusuff	SFLA	[13]
Shuffled Shepherd Optimisation	A. Kaveh	SSO	[25]
Sine Cosine Algorithm	S. Mirjalili	SCA	[38]
Social Spider Optimisation	E. Cuevas	SSPO	[8]
The Bees Algorithm	D. T. Pham	TBA	[45]
Thermal Exchange Optimisation	A. Kaveh	TEA	[20]
Vibrating Particles System	A. Kaveh	VPS	[22]
Water Strider Algorithm	A. Kaveh	WSA	[21]
Whale Optimisation Algorithm	S. Mirjalili	WOA	[42]

Table II lists all the other metaheuristics implemented, as well as the authors and abbreviated tag. Each of these represents an algorithm that may or may not be run with various control parameters, depending on whether the algorithm implements control parameters. Additionally, some algorithms appear for a second time in the results, with an asterisk attached to the tag. This indicates where the code provided by the author differs enough from the algorithm described in their article that it was deemed necessary to implement both versions of the metaheuristic. In these cases, the algorithm implemented from the code provided by the author is tagged with the asterisk.

C. Experimental Process

Each metaheuristic detailed in Section IV-B is run against every benchmark function used with potentially many different parameter sets, based on whether the metaheuristic takes control parameters. For those metaheuristics that take control parameters, the control parameters are determined on an

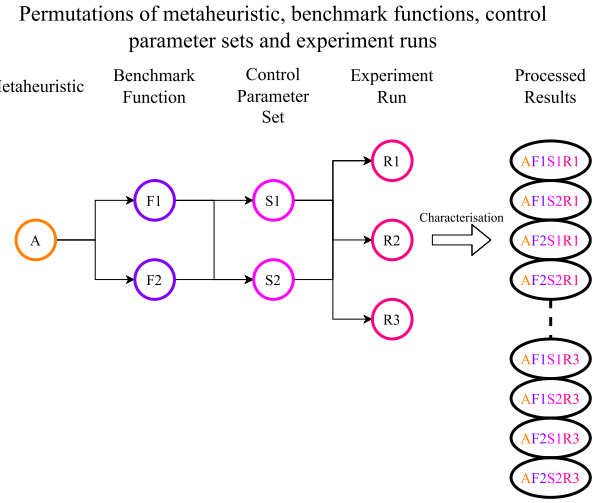

Fig. 2. Visual representation of how metaheuristics, benchmark functions, control parameter sets, and experiment runs are combined and processed through characterization to create many sets of results.

individual basis, depending on the recommendation of the original author. All experiments are run with 30 individuals in the algorithm's population. While some algorithms recommend different number of individuals, by maintaining the same population size across all algorithms, the number of uncontrolled variables when comparing algorithmic behavior is reduced.

Each experiment consists of 10 independent runs of the algorithm. Starting positions for the initial population are fixed for each run, so that there are 10 sets of fixed starting positions for each benchmark function. This is done so that all algorithms are run with the same starting conditions, while maintaining a level of variety. This removes an element of stochasticity from the results and allows for fair comparison between algorithms. Multiple runs of the same algorithm instance are required to allow for a single algorithm instance to exhibit different behaviors. Due to the stochastic nature of metaheuristics, it is plausible that different runs of the same algorithm instance may vary considerably.

In order to calculate the individual behavioral characteristics, various indicators are logged. To create the IN, every social interaction in the optimisation process is logged by increasing the social tally between the individuals taking part in the interaction. To create the STN, every individual in the optimisation process logs their position trajectory every number of iterations, along with the change in fitness value. Diversity and accuracy measures are logged once every iteration. At the end of each run, the total objective function evaluations are logged. Results are logged for every run with the metaheuristic, benchmark function and control parameter set as labels.

The logged data for each permutation of metaheuristic, benchmark function, control parameter set, and run is processed into the behavioral characteristics, after which each run of the algorithm instance is represented by a single 20-dimension behavioral vector. An example of these permutations can be seen in Fig. 2.

Finally, the processed results are clustered using a popular clustering algorithm, i.e., DBSCAN [12]. This specific algorithm is chosen as it allows for explicit outliers. For the analysis done in this article, outliers represent an important factor, which is uniqueness. Algorithms which cannot be clustered suggest unique behavior. The results for each benchmark function form a subset, on which clustering is performed; additionally, clustering for each behavioral category identified in Section II is performed for every function's subset of results. By clustering the processed results, behavioral profiles are identified. Instances of an algorithm clustered together exhibit the same behavioral profile for the given behavioral category. By maintaining each processed run of an algorithm instance as an independent data point during clustering, it allows for a single algorithm instance to exhibit multiple behavioral profiles. Multiple runs of an algorithm instance found in the same cluster are deduplicated once the clustering process is complete. This process is illustrated in Fig. 3.

In order to be able to quantify the similarity of metaheuristics across multiple functions, acknowledging that search behavior is problem dependant, the following method is used. For every metaheuristic tested, a pair-wise relationship is formed with every other metaheuristic, with an initial value of zero. For every function, every cluster created is iterated over, identifying how many times a unique metaheuristic is found in the same cluster as another unique metaheuristic. Each time such a pairing is found, the pair-wise relationship established between the two involved metaheuristics is increased by one. By doing so, the number of times that a pair of metaheuristics are found within the same behavioral profile is tallied. Once all pairs have been tallied, the relationships can be illustrated using a square heatmap, which has been dubbed a frequency map. This allows for easy, visual analysis of the distinctness of any metaheuristic's behavior, as well as where its similarities may lie. This procedure is followed for the clustering of the complete 20-dimension result vectors, as well as for the clustering of each behavioral group.

The results of the clustering are discussed in Section V.

D. Limitations

Due to the stochastic nature of metaheuristics, there are known limitations to this approach. There is an assumption made that running the same algorithm on the same problem, with the same control parameters and starting positions, will generate the same algorithmic behavior. This is not necessarily true, because many metaheuristics continually introduce randomness during the optimisation process. This limitation is hopefully mitigated by increasing the number of runs for each given metaheuristic-parameter set permutation, so that the likelihood of observing all behavior profiles of the permutation is high.

A second limitation is the trivial nature of the optimisation problems chosen. Papers proposing new algorithms typically include harder, nontrivial optimisation problems, such as real-world problems from the engineering discipline. While this study does not aim to comment on the effectivity of one algorithm over another, considering the impact a harder

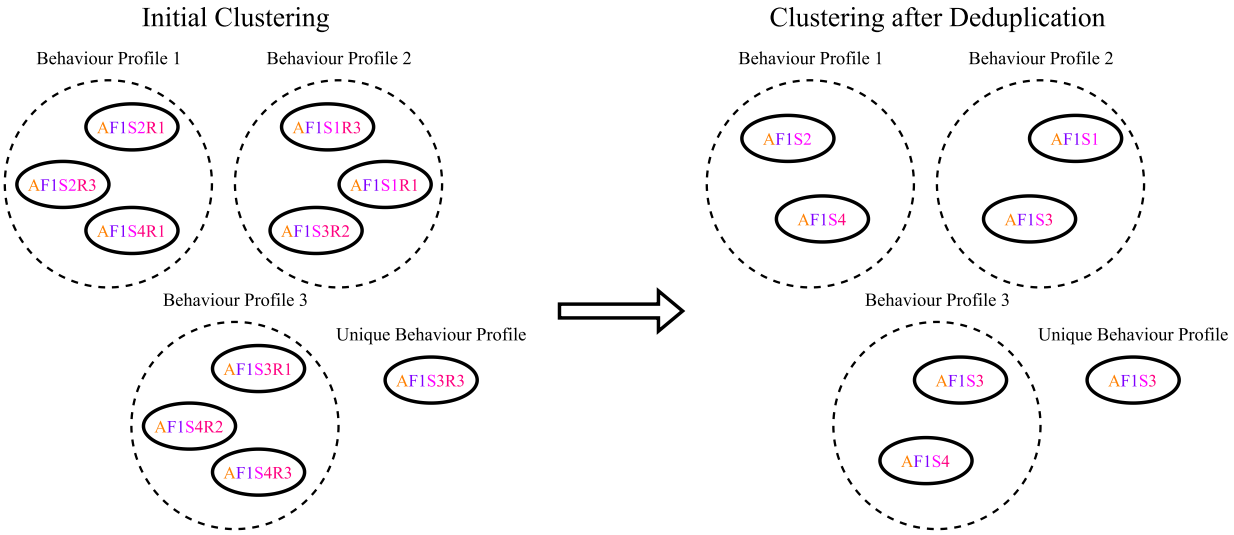

Fig. 3. Visualization of the clustering process. Initially the processed results for a single benchmark function are clustered, with different runs of the algorithm instance being independent data points. Data points sharing a cluster indicate that the algorithm instances of the data points share a behavioral profile. Data points that are outliers indicate the algorithm instance of the data point has a unique behavioral profile. Deduplication then takes place, reducing multiple runs of an algorithm instance to a single data point. This is done for the purposes of creating a frequency map.

problem may have on algorithmic behavior is important. It is possible that some algorithms' behaviors are drastically different on harder problems, which will not be able to be observed in this article.

V. Research Results

The following section details the results and analysis of the clustering of the data generated by running metaheuristics on benchmarking functions. The results are valid for the algorithms published in the papers referenced, and it should be noted that any deviation from the described algorithm will result in different behavior, not to be compared to the results here. It should also be noted that as behavioral similarity does not necessarily mean structural similarity, algorithms which are not structurally similar may be found to be behaviorally similar.

First, the goals and aims of the analysis are given. Section V-A reports on the overall clustering results, after which follows sections reporting on the clustering of the behavioral groups. Section V-H concludes the result analysis with overarching comments.

A. Analysis Goals

In order to follow through on the goal of finding similarities among metaheuristics, a similarity must be defined. For this analysis, a similarity between two algorithms is defined as being consistently placed in the same cluster. This is tested by answering the following questions.

1) Are the metaheuristics clustered together over many different benchmark functions?
2) Are the metaheuristics clustered together for the overall clustering, as well as the individual behavioral group clustering?

If a pair of algorithms give affirmative answers for both these questions, the two algorithms will be deemed similar. Metaheuristics which are consistently not placed in any cluster are defined as having unique behavior. Because these questions deal with frequency, and are not concerned with the details of which data points are in which clusters, the results are tailored to highlight frequencies between pairs of algorithms.

B. Clustering—Overall

The first set of results to be analyzed is the overall clustering set, in which the entire 20-dimension feature vectors generated during the experiments are clustered in subsets of each of the five benchmark functions.

In order to perform frequency-based analysis on the clusters, the clustered data is summarized as follows: a frequency matrix is created, with a pairing between every metaheuristic with every other metaheuristic. Every time a metaheuristic is found within the same cluster as another metaheuristic, the frequency of that pairing is increased. The frequency matrix is completed by inspecting every unique metaheuristic in all clusters for every benchmark function. The results of this process are illustrated in Fig. 4, which includes a description of how the frequency maps are to be interpreted.

Referring to overall portion of Fig. 4, it should first be noted that, in general, most metaheuristics are frequently clustered with most other metaheuristics. While the color pattern of this frequency map is diverse, most frequencies are high, with individual metaheuristics exhibiting more unique behaviors than others. This may suggest that most metaheuristics implemented in this study are behaviorally similar.

Next, attention may be drawn to the black bands going across the frequency map. "SA," "SFLA," and "MRFO" are all classified as unique with regards to the first similarity question. It is worthwhile to note that "BA," "ABC," "SOA*," and A-EP are also largely distinct, after which "CSS" and "SSA*" follow in uniqueness.

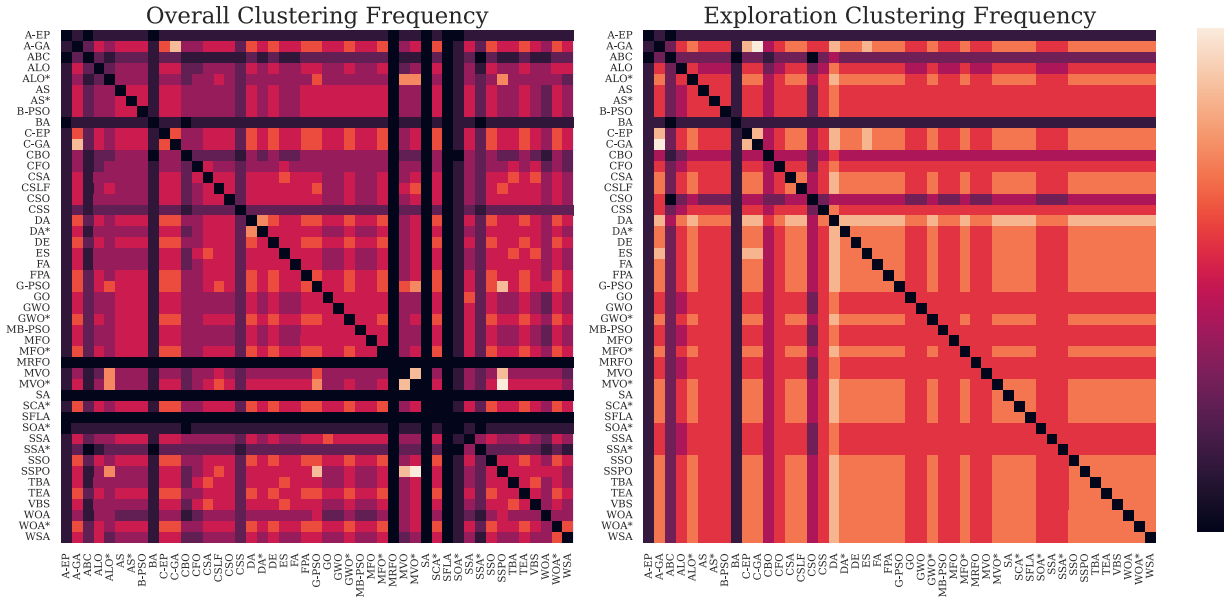

Fig. 4. Frequency of a given metaheuristic appearing in the same cluster as another metaheuristic, over all benchmark functions during the overall clustering, and the clustering for the exploration behavior division. The relationships between metaheuristics are read using the *x*-axis labels and the *y*-axis labels, with the state of the relationship being defined by the color of the block at the intersection of the coordinates. Darker colors indicate that the metaheuristics do not share behaviors and lighter colors indicate that the metaheuristics do share behaviors. A white block indicates that the two involved metaheuristics can be considered as the same, with regards to the definition of similarity, namely, that they are clustered together over many different benchmark functions. A black band indicates that a metaheuristic's behavioral profile is entirely unique, because it could not be clustered over all benchmark functions.

Finally, consider the identical metaheuristics. The "MVO*" metaheuristic is distinctly linked to "SSPO." "AO*" is closely linked to both SSPO and MVO*, indicating that together this trio of metaheuristics may overlap in behavioral profiles. SSPO also links with "G-PSO," which one might think to have some of the oldest known behaviors. This may indicate that the behaviors observed in SSPO are very basic. Additionally, the MVO* metaheuristic is very closely linked with the "MVO." This is to be expected, because these are twin algorithms, with the one being the code provided for a article, and the other the direct implementation of the pseudocode in this article. While such similarities may be expected, as seen in other examples, such as SSA* and "SSA," it is not guaranteed. In fact, while the code provided for the Salp Swarm Algorithm paper proves to exhibit unique behaviors, the algorithm described in this article exhibits different behaviors, which are not unique. It can also be seen that A-GA and C-GA exhibit closely similar behaviors. Because the two represent two classes of genetic algorithms, this makes sense.

C. Clustering—Exploration

The first division of clustering is the exploration behavior category. The values for the five behavioral characteristics under "Exploration", i.e., DRoC, Prime Spread, Accuracy Rate of Change (Types A and B), and PFV, are selected from the results and clustered using DBSCAN with tuned parameters. Using the same type of frequency map defined before, the exploration portion of Fig. 4 is created.

The initial impression is that the amount of diversity in exploration behaviors is very small. There are four distinct groups, ranging from least distinct to unique exploration behaviors.

The first of these groups is the light yellow color, indicating the metaheuristics with most similar behaviors. In this group "DA" has a light yellow band across the width of the map, indicating that it does not display any unique exploration behaviors. Similarly, as with the overall results, A-GA and C-GA are exact matches for one another. A-GA and "C-DA" differ only in that the adaptive genetic algorithm gradually reduces its mutation rate, crossover rate, and the mutation step size. The results indicate that this difference does not account for any significant difference in behavior. Additionally, A-GA, C-GA, "ES," and C-EP are all frequently paired together. As genetic algorithms, evolutionary strategies and evolutionary programming fall under evolutionary algorithms, it makes sense that behaviors are highly similar, with all three types of metaheuristics observing concepts, such as selection, mutation, and crossover of individuals. This suggests that there would be little behavioral difference selecting one of these metaheuristics over another.

The largest of these groups, the light orange color, takes up large blocks of the total map. Too many metaheuristics to mention individually, the prevalence of this group suggests that there are a few core exploration behaviors that most popular metaheuristics follow. The next group is the deep orange color. Also taking up large amounts of space on the map, with large deep orange bands crossing over the matrix. The banded pattern indicates that a few specific metaheuristics always

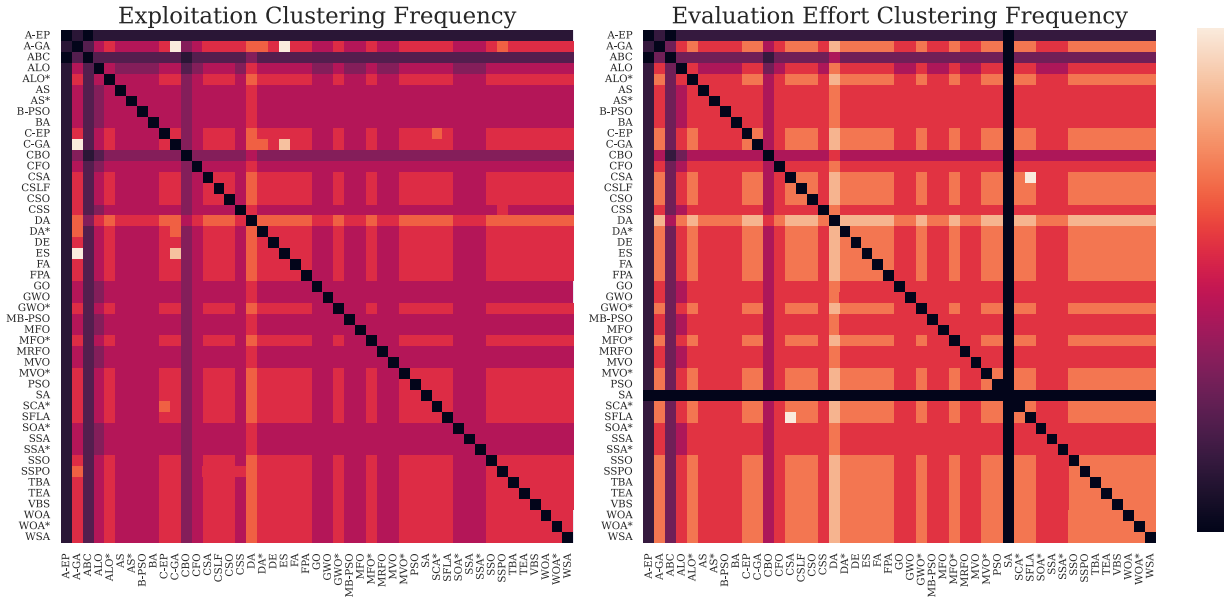

Fig. 5. Frequency of a given metaheuristic appearing in the same cluster as another metaheuristic, over all benchmark functions during the exploitation division clustering, and the evaluation division of clustering. Interpretation guide to be found in Fig. 4.

exhibit slightly more distinct exploration behaviors than the previous group.

The last group is that of unique explorative behavior. BA and A-EP lead, displaying the most number of unique behaviors, with ABC, "CSO," and "CBO" also exhibiting higher degrees of distinctiveness. Interestingly, BA, ABC and A-EP were unique in the overall clustering results as well. CSO, however, is not distinct in the overall clustering. This suggests that in terms of exploration, the CSO metaheuristic exhibits unique behaviors, but that the rest of its behavioral profile is not similarly distinct.

D. Clustering: Exploitation

The next division of clustering is the exploitation behavior category. The values for the four behavioral characteristics under "Exploitation", i.e., CRoC, Locality Rate of Change (Types A and B), and ER, are selected from the results and clustered using DBSCAN with tuned parameters. Using the defined frequency map, the exploitation portion of Fig. 5 is created.

The color pattern profile of the exploitation portion of Fig. 5 is very uniform. While this may indicate that the exploitative behavior characteristics are not satisfyingly differentiating between exploitation behaviors, it may also be that there are not many different ways and algorithm may exploit. As such, there is little diversity in exploitative behaviors exhibited, and in turn little diversity in the frequency map.

There are still some interesting conclusions which can be made. It can be seen that yet again, A-GA, C-GA and ES are very frequently paired. It is becoming clear that this grouping of algorithms are indistinct from one another. DA creates a light orange band across the map, indicating that overall its exploitative behaviors are least distinct of all. Similarly, under the exploration division it was also markedly indistinct. The most distinct metaheuristics are ABC and A-EP, with A-EP being almost unique.

E. Clustering: Evaluation Effort

The next division of clustering is the evaluation effort behavior category. The five values of the behavioral characteristics under 'Evaluation Effort', i.e., Objective Function Evaluations, ERT-Diversity, ERT-Accuracy-A, ERT-Accuracy-B, %INFEASIBLE, are selected from the results and the evaluation effort portion of Fig. 5 is created based on clustering using DBSCAN with tuned parameters.

Fig. 5's evaluation effort map looks very similar to the overall frequency map of Fig. 4, having large blocks of light orange and bands of dark orange crossing over the map. Starting with least distinct, 'SFLA' and 'CSA' are exact matches for one another. As before, DA creates a light yellow band across the frequency map, indicating that it shows no unique behaviors with regards to evaluation effort.

SA and A-EP shows highly distinct evaluation effort behavioral profiles. SA is unlike other metaheuristics, because it only generates a single new point each iteration, leading to far fewer objective function evaluations. Based on the control parameter set, A-EP adjusts the mutation step sizes based on fitness value, leading to an unusual number of objective function evaluations.

F. Clustering—Communication

The next division of clustering is the communication behavior category. The four values of the behavioral characteristics under 'Communication', i.e., BSoI, IRoF, best-strength and nbest, are selected from the results and the communication portion of Fig. 6 is created based on clustering using DBSCAN with tuned parameters.

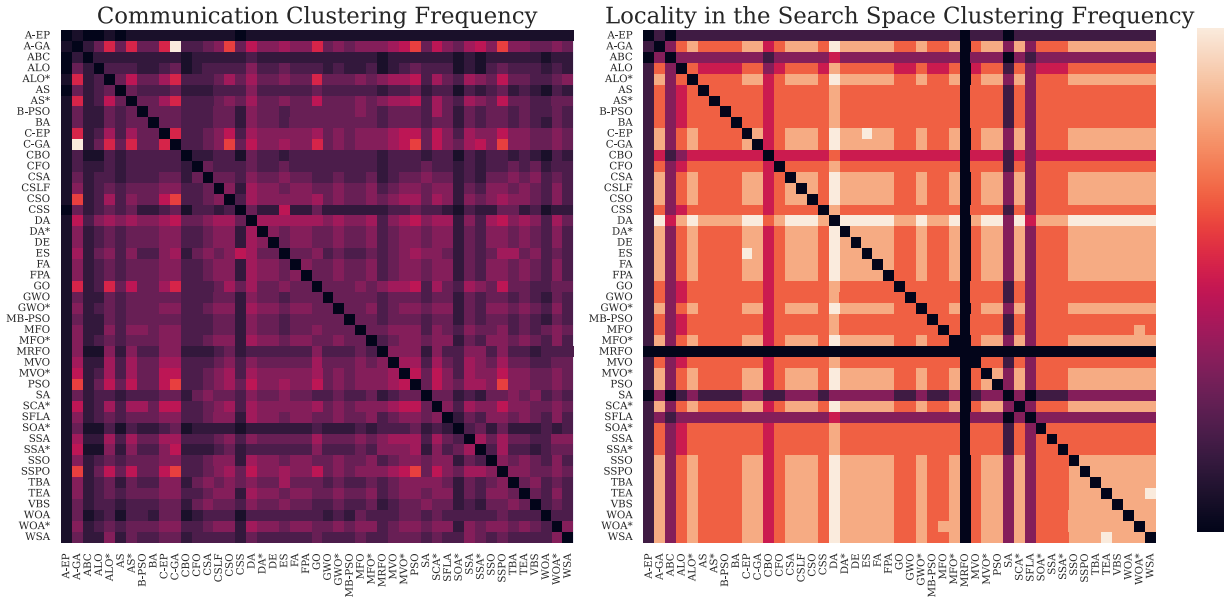

Fig. 6. Frequency of a given metaheuristic appearing in the same cluster as another metaheuristic, over all benchmark functions during the communication division clustering, and the division for locality in the search space. Interpretation guide to be found in Fig. 4.

The communication frequency map in Fig. 6 is the least diverse of all the frequency maps seen so far. This map, in opposition to the exploitation frequency map, has a very high rate of distinct behaviors. This suggests that communication characteristics are good to have in the overall clustering, because they are effective at distinguishing metaheuristics from each other. However, standing alone, it does not make a good tool for analysis. Various reasons for the lack of diversity can be suggested. One hypothesis is that due to the large number of individuals in a population, there are many ways to share information. Another hypothesis is that communication is less critical for an algorithm's success, allowing for greater freedom designing methods of communication when designing an algorithm.

As in every frequency map seen before, A-GA and C-GA are exact matches for one another. While most metaheuristics are distinct, the bands for C-GA and A-GA show that these two algorithms are more frequency paired with other metaheuristics than most. This indicates that the underlying communication behaviors are already present in the original evolutionary algorithms. Because most metaheuristics in this map are distinct, is not worth mentioning them. Only A-EP will be mentioned, as the black band across the map indicates it is entirely unique.

G. Clustering—Locality in Search Space

The last division of clustering is the locality in the search space behavior category. The three values of the behavioral characteristics under 'Locality in Search Space', i.e., nshared, ntotal, nend, are selected and the locality in the search space portion of Fig. 6 is created based on clustering using DBSCAN with tuned parameters.

As the last frequency map to be analyzed, the analysis comes full circle, because the patterns in the right frequency map in Fig. 6 largely reflect the patterns in the overall clustering in Fig. 4. DA matches exactly with almost all other metaheuristics, confirming that DA contains no unique behaviors. C-EP is completely paired with ES, which goes with the trend of evolutionary algorithms being strongly paired. 'WSA' is exactly paired with 'TEA'. Both algorithms follow position update strategies which allow individuals to be attracted to the positions of potentially many different other individuals, creating a potentially more diverse search pattern. As seen before, unique metaheuristics with regards to the locality in search space are 'MRFO', followed by SA, ABC, 'SFLA' and A-EP. All of these unique metaheuristics have been identified as unique before, solidifying the identification of them as such.

H. Similarity Conclusion

Having now identified similar and dissimilar metaheuristics across both overall clustering and divisioned clustering, conclusions can be made about which metaheuristics are behaviorally unique, which are not, and which are completely indistinct from all other metaheuristics.

The most consistently unique metaheuristic of all is 'AdaptiveEP', which is an adaptive evolutionary program. While an original evolutionary algorithm, the behaviors it exhibits cannot be found in any other metaheuristics. This is in contrast to other evolutionary algorithms, such as genetic algorithms and evolutionary strategies, such as A-GA, C-GA, and ES, which are frequently paired with all other metaheuristics. Additional analysis of adaptive evolutionary programs would be required in order to understand what makes their behavior so distinct.

A consistently unique metaheuristic is SA. This serves as a correctness test for the behavioral characteristic suite, as well as the clustering results. This is because simulated annealing is different to all the other metaheuristics implemented, as it is not population based. This difference impacts every aspect

of its behavior profile. By being marked distinct, we confirm that the methodology used in this article is correct.

ABC is frequently marked as unique. The algorithm of ABC is very complex, containing multiple position update steps and branching logic. This complexity is likely what distinguishes it from the other metaheuristics. This uniqueness comes at the price of not being easily understood. The algorithm is heavily laden with metaphor, which makes it difficult to implement, discouraging its use.

Other metaheuristics may not be unique across all divisions, but combinations of behaviors are unique. Examples are BA, 'SOA*', 'SFLA' and 'MRFO', which are all distinct in the overall clustering and in one or two of the division clusterings. Most interesting of these is 'SOA*', which is perhaps unique in that it is ineffective. The algorithm provided in this article is completely contrary to the code provided by the author, and even then the code is confusing and ineffective at optimizing simple benchmark functions. This is a good reminder that distinct behaviors are not necessarily positive.

Least distinct is DA, which rarely exhibits any unique behaviors. The algorithm described in DA is highly complicated and yet provides no new behaviors. This serves to remind that algorithmic behaviors are not created by throwing enumerable terms into every equation, but rather through calculated thought and intention.

VI. Conclusion

In order to quantify metaheuristic search behavior and to find similarities and dissimilarities between metaheuristics, a behavioral characteristic benchmarking suite is proposed. The benchmarking suite quantifies search behavior in terms of how an algorithm explores the search landscape, how it exploits the search landscape, how it shares information via communication, the areas of the search landscape that it visits, and the amount of effort that it puts in to solving the optimisation problem. Using the benchmarking suite, as well as a host of some of the most popular metaheuristics, experiments were conducted. The experiments consisted of running the metaheuristics on optimisation problems, while using many different control parameter sets, and for multiple runs, while collecting data about the search process. The benchmark characteristics were calculated using the data logged during experimentation. The processed results were clustered using DBSCAN, and the results of clustering were further processed by calculating frequency maps. The frequency maps are designed to indicate patterns of the frequency of similarities and dissimilarities between pairs of algorithms.

Analysis of the frequency maps leads to a number of conclusions. The first, and most over-arching, is that most metaheuristics share their search behaviors with most other metaheuristics. This conclusion is motivated by the low number of unique behaviors indicated by the frequency maps of the behavioral categories. This conclusion indicates that many newly published metaheuristics are unique only in their inspiration, and that the search behaviors are not significantly different to existing algorithms. Additionally, this allows for it to be concluded that unique metaheuristics are more likely to be unique combinations of preexisting search behaviors. This is motivated by the number of unique metaheuristics indicated by the overall clustering frequency map, which is a greater number than that of any of the frequency maps of the behavioral categories. This can be used to the advantage of authors creating new metaheuristics. Acknowledging that the pool of search behaviors is limited, algorithms can be crafted for specific purposes by combining select behaviors from the behavioral categories, in order to best solve the given problem.

A second conclusion is that most search behaviors exhibited overall are also exhibited by the baseline original metaheuristics. Given that the baseline algorithms are decades old, in comparison to the newer metaheuristics implemented which are at times only a few years old, this indicates that the pattern of publishing algorithms which introduce no novelty is an old pattern. This conclusion is based on the frequency with which the baseline algorithms paired with all other metaheuristics across all frequency maps considered. This suggests that the behavior profiles of most metaheuristics could be recreated using a version of a baseline algorithm. This motivates future work in understanding the behaviors of specific baseline algorithms and the influence that specific operators and parameters have on those behaviors.

It is also concluded that small changes to an algorithm may make for large changes in behavior. With regards to the twin algorithms implemented in the metaheuristic suite, the small changes between the twin algorithms results in different behavioral profiles being exhibited. Algorithmic steps may be omitted in this article to save space, or confusing language may be used, which results in the implemented algorithm having different behavior than the author intended. This motivates for higher standards to be enforced when publishing new algorithms.

In conclusion, the results confirm the hypothesis that many newly published metaheuristics do not offer novelty beyond their nature-inspired metaphors.

Potential future work includes rigorously testing the assumptions made. These assumptions specifically include the fixing of the population size and the dimension of the benchmarking functions, the effect of computational budget assigned to experiments, and how the characteristic benchmarking suite performs in dynamic environments or on self-adaptive algorithms. With regards to the fixing of the population size and dimension of benchmarking functions, investigating how these factors influence and change algorithm behavior is important in establishing if the results and conclusions hold universally. Similarly, asserting whether changing the computational budget changes the behavioral characteristics of algorithms tests whether the characteristic benchmarking suite is robust. Further experimenting with dynamic benchmark functions and self-adaptive metaheuristics will determine how widely applicable the characteristic benchmarking suite and methodology are.

Other future work would be implementing techniques through which behavioral ranges may be established. As previously noted, algorithms may exhibit different behaviors with different control parameter sets. The characteristic benchmarking suite could be used to identify for which ranges of

control parameters algorithms exhibit specific behaviors, and when behavioral transitions occur.

Future work would also be investigating the impact of a different suite of benchmarking problems, as well as harder benchmarking problems. Comparing the results obtained on a different benchmarking suite would test whether the results are transferable, and whether the conclusions that have been made hold true. Testing using harder problems would allow for the behavioral analysis and clustering to be able to identify specific behaviors that indicate good performance on harder problems. This would also allow for analysis of changing algorithm behavior as benchmark functions become harder to solve.

Other potential future would could be investigating the relationship between algorithm search behavior and other areas of algorithms classification, such as according to performance or the presence of structural bias [50]. Specifically, the aim would be to identify whether certain factors indicate others, such as if certain behaviors indicate if structural bias is present.

Finally, other future work involves investigating the characteristic benchmark suite, specifically to find which characteristics are most impactful, and where there is redundancy. The characteristic benchmarking suite could potentially be reduced while maintaining the same level of characterization, which would reduce the computational expense of calculating the characteristics.

REFERENCES

[1] H. A. Abbass, "MBO: Marriage in honey bees optimization-a haplometrosis polygynous swarming approach," in *Proc. Congr. Evol. Comput.*, vol. 1, 2001, pp. 207–214.

[2] C. Aranha et al., "Metaphor-based metaheuristics, a call for action: The elephant in the room," *Swarm Intell.*, vol. 16, no. 1, pp. 1–6, 2022.

[3] P. Bosman and A. P. Engelbrecht, "Diversity rate of change measurement for particle swarm optimisers," in *Proc. Int. Conf. Swarm Intell.*, 2014, pp. 86–97.

[4] F. Campelo and C. Aranha. "EC bestiary: A bestiary of evolutionary, swarm and other metaphor-based algorithms." 2018. [Online]. Available: http://fcampelo.github.io/EC-Bestiary/

[5] S.-C. Chu, P. Tsai, and J.-S. Pan, "Cat swarm optimization," in *Proc. Pac. Rim Int. Conf. Artif. Intell.*, 2006, pp. 854–858.

[6] CW. Cleghorn and A. P. Engelbrecht, "Particle swarm variants: Standardized convergence analysis," *Swarm Intell.*, vol. 9, no. 2, pp. 177–203, 2015.

[7] E. Cuevas et al., "Search patterns based on trajectories extracted from the response of second-order systems," *Appl. Sci.*, vol. 11, no. 8, p. 3430, 2021.

[8] E. Cuevas, M. Cienfuegos, D. Zaldívar, and M. Pérez-Cisneros, "A swarm optimization algorithm inspired in the behavior of the social-spider," *Exp. Syst. Appl.*, vol. 40, no. 16, pp. 6374–6384, 2013.

[9] G. Dhiman and V. Kumar, "Seagull optimization algorithm: Theory and its applications for large-scale industrial engineering problems," *Knowl.-Based Syst.*, vol. 165, pp. 169–196, Feb. 2019.

[10] A. E. Eiben and M. Jelasity, "A critical note on experimental research methodology in EC," in *Proc. Congr. Evol. Comput. (CEC)*, vol. 1, 2002, pp. 582–587.

[11] A. Engelbrecht, P. Bosman, and K. Malan, "The influence of fitness landscape characteristics on particle swarm optimisers," *Natural Comput.*, vol. 21, no. 2, pp. 335–345, 2022.

[12] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discov. Data Mining*, vol. 96, 1996, pp. 226–231.

[13] M. Eusuff, K. Lansey, and F. Pasha, "Shuffled frog-leaping algorithm: A memetic meta-heuristic for discrete optimization," *Eng. Optim.*, vol. 38, no. 2, pp. 129–154, 2006.

[14] L. J. Fogel, "Autonomous automata," *Ind. Res.*, vol. 4, no. 2, pp. 14–19, 1962.

[15] R. A. Formato, *Central Force Optimization: A New Nature Inspired Computational Framework for Multidimensional Search and Optimization*. Heidelberg, Germany: Springer, 2008. pp. 221–238.

[16] A. S. Fraser, "Simulation of genetic systems by automatic digital computers," *Aust. J. Biol. Sci.*, vol. 10, no. 4, pp. 484–491, 1957.

[17] J. Hooker, "Testing heuristics: We have it all wrong," *J. Heuristics*, vol. 1, pp. 33–42, Sep. 1995.

[18] K. Hussain, M. N. Mohd Salleh, S. Cheng, and Y. Shi, "Metaheuristic research: A comprehensive survey," *Artif. Intell. Rev.*, vol. 52, no. 4, pp. 2191–2233, 2019.

[19] D. Karaboga and B. Basturk, "Artificial bee colony (ABC) optimization algorithm for solving constrained optimization problems," in *Proc. Int. Fuzzy Syst. Assoc. World Congr.*, 2007, pp. 789–798.

[20] A. Kaveh and A. Dadras, "A novel meta-heuristic optimization algorithm: Thermal exchange optimization," *Adv. Eng. Softw.*, vol. 110, pp. 69–84, Aug. 2017.

[21] A. Kaveh and A. Dadras Eslamlou, "Water strider algorithm: A new metaheuristic and applications," *Structures*, vol. 25, pp. 520–541, Jun. 2020.

[22] A. Kaveh and M. I. Ghazaan, "A new meta-heuristic algorithm: Vibrating particles system," *Scientia Iranica. Trans. A, Civil Eng.*, vol. 24, no. 2, pp. 551–566, 2017.

[23] A. Kaveh and V. R. Mahdavi, "Colliding bodies optimization: A novel meta-heuristic method," *Comput. Struct.*, vol. 139, pp. 18–27, Jul. 2014.

[24] A. Kaveh and S. Talatahari, "A novel heuristic optimization method: Charged system search," *Acta Mechanica*, vol. 213, no. 3, pp. 267–289, 2010.

[25] A. Kaveh and A. Zaerreza, "Shuffled shepherd optimization method: A new meta-heuristic algorithm," *Eng. Comput.*, vol. 37, no. 7, pp. 2357–2389, 2020.

[26] J. Kennedy, "Bare bones particle swarms," in *Proc. IEEE Swarm Intell. Symp. (SIS)*, 2003, pp. 80–87.

[27] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. Int. Conf. Neural Netw. (ICNN)*, vol. 4, 1995, pp. 1942–1948.

[28] S. Kirkpatrick, C. Gelatt, and M. Vecchi, "Optimization by simulated annealing," *science*, vol. 220, no. 4598, pp. 671–680, 1983.

[29] J. R. Koza, "Genetic programming as a means for programming computers by natural selection," *Statist. Comput.*, vol. 4, no. 2, pp. 87–112, 1994.

[30] K. N. Krishnanand, "Glowworm swarm optimization for simultaneous capture of multiple local optima of multimodal functions," *Swarm Intell.*, vol. 3, no. 2, pp. 87–124, 2009.

[31] R. D. Lang and A. P. Engelbrecht, "An exploratory landscape analysis-based benchmark suite," *Algorithms*, vol. 14, no. 3, p. 78, 2021.

[32] X. Meng, Y. Liu, X. Gao, and H. Zhang, "A new bio-inspired algorithm: Chicken swarm optimization," in *Advances in Swarm Intelligence*, Y. Tan, Y. Shi, and C. A. Coello Coello, Eds. Hefei, China: Springer, 2014, pp. 86–94.

[33] O. Mersmann, B. Bischl, H. Trautmann, M. Preuss, C. Weihs, and G. Rudolph, "Exploratory landscape analysis," in *Proc. 13th Annu. Conf. Genet. Evol. Comput.*, 2011, pp. 829–836.

[34] O. Mersmann, M. Preuss, and H. Trautmann, "Benchmarking evolutionary algorithms: Towards exploratory landscape analysis," in *Proc. Int. Conf. Parallel Probl. Solv. Nat.*, 2010, pp. 73–82.

[35] S. Mirjalili, "The ant lion optimizer," *Adv. Eng. Softw.*, vol. 83, pp. 80–98, May 2015.

[36] S. Mirjalili, "Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm," *Knowl.-Based Syst.*, vol. 89, pp. 228–249, Nov. 2015.

[37] S. Mirjalili, "Dragonfly algorithm: A new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems," *Neural Comput. Appl.*, vol. 27, no. 4, pp. 1053–1073, 2016.

[38] S. Mirjalili, "SCA: A sine cosine algorithm for solving optimization problems," *Knowl.-Based Syst.*, vol. 96, pp. 120–133, Mar. 2016.

[39] S. Mirjalili, A. H. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, "Salp swarm algorithm: A bio-inspired optimizer for engineering design problems," *Adv. Eng. Softw.*, vol. 114, pp. 163–191, Dec. 2017.

[40] S. Mirjalili and A. Hatamlou, "Multi-verse optimizer: A nature-inspired algorithm for global optimization," *Neural Comput. Appl.*, vol. 27, no. 2, pp. 495–513, 2016.

[41] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Adv. Eng. Softw.*, vol. 69, pp. 46–61, Mar. 2014.

[42] S. Mirjalili and A. Lewis, "The whale optimization algorithm," *Adv. Eng. Softw.*, vol. 95, pp. 51–67, May 2016.

[43] G. Ochoa, K. Malan, and C. Blum, "Search trajectory networks: A tool for analysing and visualising the behaviour of metaheuristics," *Appl. Soft Comput.*, vol. 109, Sep. 2021, Art. no. 107492.
[44] M. Oliveira, D. Pinheiro, M. Macedo, C. Bastos-Filho, and R. Menezes, "Uncovering the social interaction network in swarm intelligence algorithms," *Appl. Netw. Sci.*, vol. 5, no. 1, p. 24, 2020.
[45] D. T. Pham, A. Ghanbarzadeh, E. Koç, S. Otri, S. Rahim, and M. Zaidi, "The bees algorithm—A novel tool for complex optimisation problems," in *Intelligent Production Machines and Systems*, D. T. Pham, E. E. Eldukhri and A. J. Soroka, Eds. Amsterdam, The Netherlands: Elsevier, 2006, pp. 454–459.
[46] I. Rechenberg, *Evolution Strategy: Optimization of Technical Systems by Means of Biological Evolution*, vol. 104. Stuttgart, Germany: Fromman-Holzboog, 1973, pp. 15–16.
[47] D. Schuurmans and F. Southey, "Local search characteristics of incomplete SAT procedures," *Artif. Intell.*, vol. 132, no. 2, pp. 121–150, Nov. 2001.
[48] K. Sörensen, "Metaheuristics—The metaphor exposed," *Int. Trans. Oper. Res.*, vol. 22, no. 1, pp. 3–18, 2015.
[49] R. Storn and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *J. Glob. Optim.*, vol. 11, no. 4, pp. 341–359, 1997.
[50] D. Vermetten, B. van Stein, F. Caraffini, L. L. Minku, and A. V. Kononova, "BIAS: A toolbox for benchmarking structural bias in the continuous domain," *IEEE Trans. Evol. Comput.*, vol. 26, no. 6, pp. 1380–1393, Dec. 2022.
[51] X. Yang and A. H. Gandomi, "Bat algorithm: A novel approach for global engineering optimization," *Eng. Comput.*, vol. 29, no. 5, pp. 464–483, 2012.
[52] X. Yang, "Firefly algorithms for multimodal optimization," in *Stochastic Algorithms: Foundations and Applications*, O. Watanabe and T. Zeugmann, Eds. Heidelberg, Germany: Springer, 2009, pp. 169–178.
[53] X. Yang, "Flower pollination algorithm for global optimization," in *Unconventional Computation Natural Computation*, J. Durand-Lose and N. Jonoska, Eds. Heidelberg, Germany: Springer, 2012, pp. 240–249.
[54] X.-S. Yang and S. Deb, "Cuckoo search via levy flights," in *Proc. World Congr. Nat. Biol. Inspired Comput.*, 2009, pp. 210–214.
[55] A. C. Zecchin, A. R. Simpson, H. R. Maier, A. Marchi, and J. B. Nixon, "Improved understanding of the searching behavior of ant colony optimization algorithms applied to the water distribution design problem," *Water Resour. Res.*, vol. 48, no. 9, 2012, Art. no. W09505.
[56] W. Zhao, L. Wang, and Z. Zhang, "Atom search optimization and its application to solve a hydrogeologic parameter estimation problem," *Knowl.-Based Syst.*, vol. 163, pp. 283–304, Jan. 2019.
[57] W. Zhao, Z. Zhang, and L. Wang, "Manta ray foraging optimization: An effective bio-inspired optimizer for engineering applications," *Eng. Appl. Artif. Intell.*, vol. 87, Jan. 2020, Art. no. 103300.

Lauren Hayward received the first B.Sc. degree and the second B.Sc. (Hons.) degrees in computer science from the University of Stellenbosch, Stellenbosch, South Africa, in 2021 and 2022, respectively.

Her research interests are in nature-inspire metaheuristics and algorithm behavior analysis.

Andries Engelbrecht (Senior Member, IEEE) received the master's and Ph.D. degrees in computer science from the University of Stellenbosch, Stellenbosch, South Africa, in 1994 and 1999, respectively.

He is a Voigt Chair of Data Science with the Department of Industrial Engineering, with a joint appointment as a Professor with the Computer Science Division, Stellenbosch University, Stellenbosch. He is the author of two books, *Computational Intelligence: An Introduction* and *Fundamentals of Computational Swarm Intelligence*. His research interests include swarm intelligence, evolutionary computation, artificial neural networks, artificial immune systems, and the application of these computational intelligence paradigms to data analytics, games, bioinformatics, finance, and difficult optimization problems.

Dr. Engelbrecht serves as an associate editor/editoral board member of a number of international journals and as a Deputy-Editor-in-Chief of the *Engineering Applications of Artificial Intelligence Journal*.